

Field Asset Health Monitor

Predictive Maintenance on Industrial Sensor Data

Figure Cover



MetroPT-3 air production unit in service — the metro train whose compressor this pipeline monitors.

A time-series anomaly-detection pipeline forecasting
air-compressor failures before they occur

MetroPT-3 Metro Train Air Production Unit

1.5M sensor readings · 7 months · 4 documented failures

1 Objective

Industrial compressors fail. The question this project answers is not *did it fail* — that is trivial after the fact — but **can we warn an engineer hours before it fails**, from noisy sensor data, reliably enough to schedule maintenance instead of suffering a breakdown.

The dataset is the MetroPT-3 air production unit of a metro train: 1,516,948 readings at a 10-second cadence across seven months (February–September 2020), seven analog and eight digital signals, and four documented air-leak failures. The deliverable is an honest, per-failure answer to how much warning each failure gives — and a runnable pipeline that produces it.

Why this is the hard version

Predicting a failure *before* it manifests is fundamentally different from detecting one while it happens. The project is built around four disciplines that separate honest prediction from after-the-fact description:

- **Train on healthy data only** — four failures cannot supervise a classifier; failures are evaluation targets, never training data.
- **Time-ordered splits, never random** — adjacent windows are near-duplicates, so shuffling would leak the answer.
- **Label the lead-up, not the failure window** — the model learns to warn, not to describe an already-broken machine.
- **Evaluate per failure** — the four failures have distinct, multidirectional signatures.

The modest AUCs are the honest cost of solving the real problem, not a shortfall to be papered over.

2 Data & Methodology

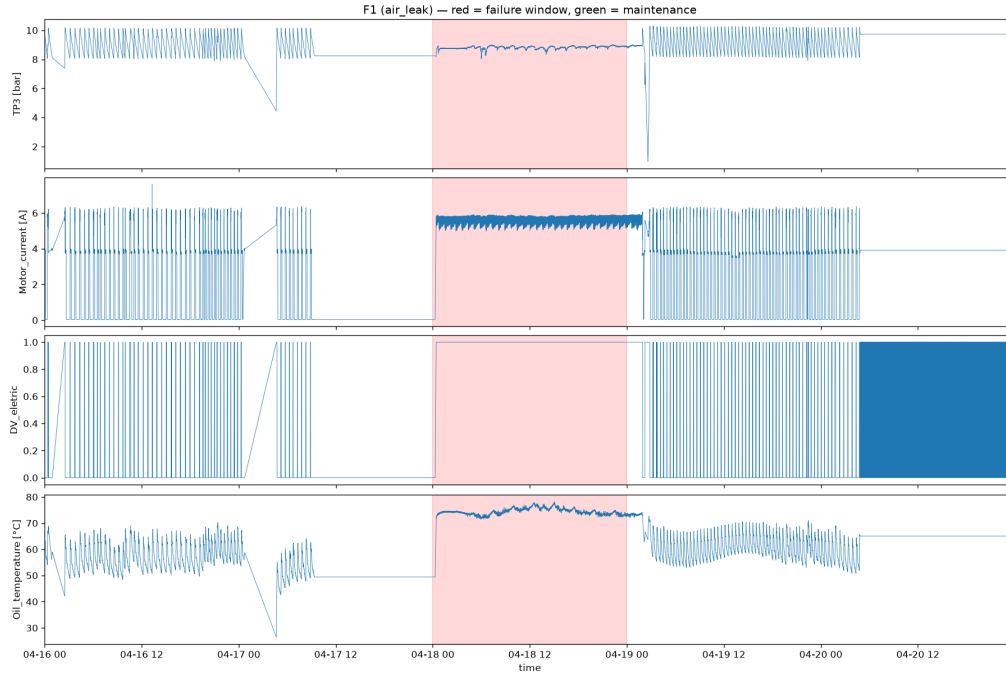
The pipeline runs in five stages, each reading and writing a data artifact so the whole run is reproducible from a single command.

Stage	In & Out	What it does
Preprocess	raw CSV & parquet	typed load, 6 integrity checks, failure windows
EDA	parquet	distributions, correlations, state-mixture analysis
Anomaly context	parquet	per-failure portraits, 6-category labeling
Features	& features.parquet	gap-aware 1h grid, 6 feature families, scaling
Modeling	& scores + report	anomaly detection, per-failure evaluation

Preprocessing & integrity

A typed load cut memory from 224 MB to 104 MB. Six integrity checks — which raise on bad data, halting the pipeline — verify row counts, timestamp monotonicity, value ranges, and the failure-window sources. A gap inventory found 331 recording gaps totalling ~18% missing time, a fact that shapes every downstream windowing decision.

Figure 2.1

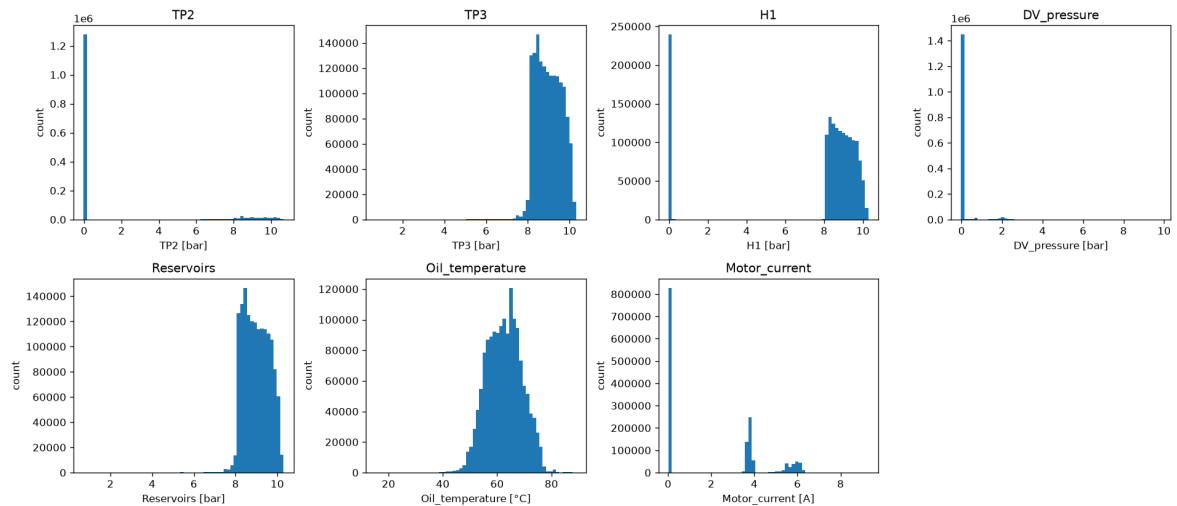


Failure portrait (F1): the sensor context around a documented failure — TP3, motor current, DV_electric, oil temperature. Red = failure window, green = maintenance.

Exploratory data analysis

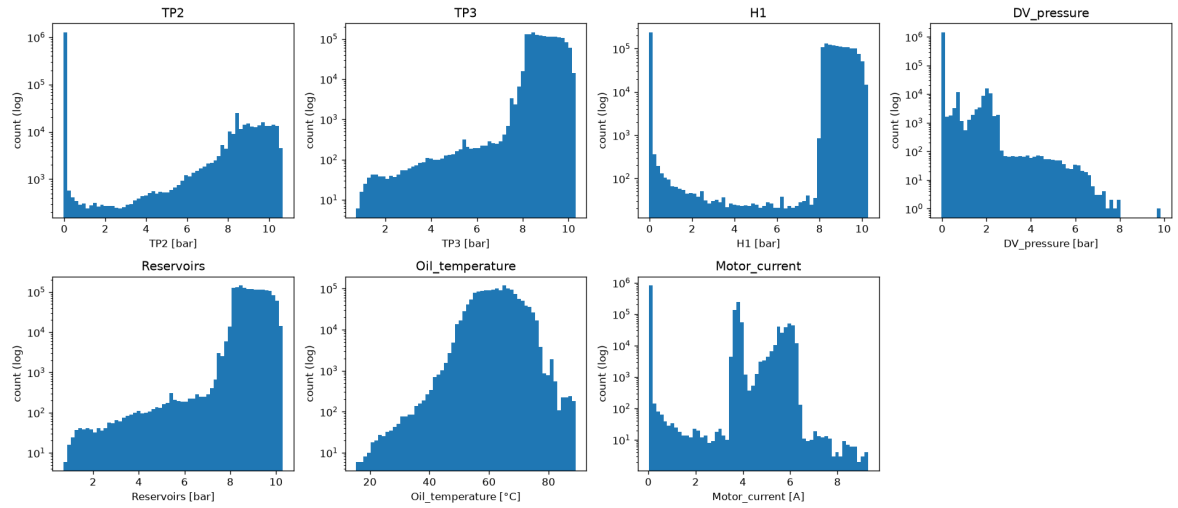
The seven analog signals are strongly bimodal — a large idle mass at zero plus an active-operation lobe — so distributions are shown on both linear and log count scales to make the sparse active tail legible. Effect sizes rather than p-values are used throughout: with 1.5M autocorrelated samples, statistical significance is meaningless, and the documented motor-current modes were corrected against the data.

Figure 2.2



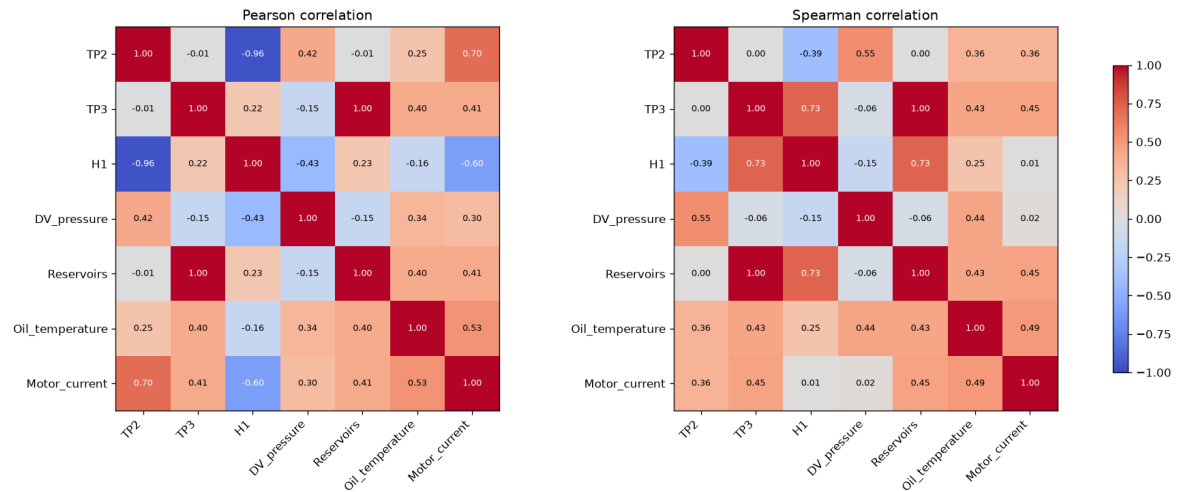
Analog signal distributions (linear count scale): TP2, TP3, H1, DV_pressure, Reservoirs, Oil_temperature, Motor_current.

Figure 2.3



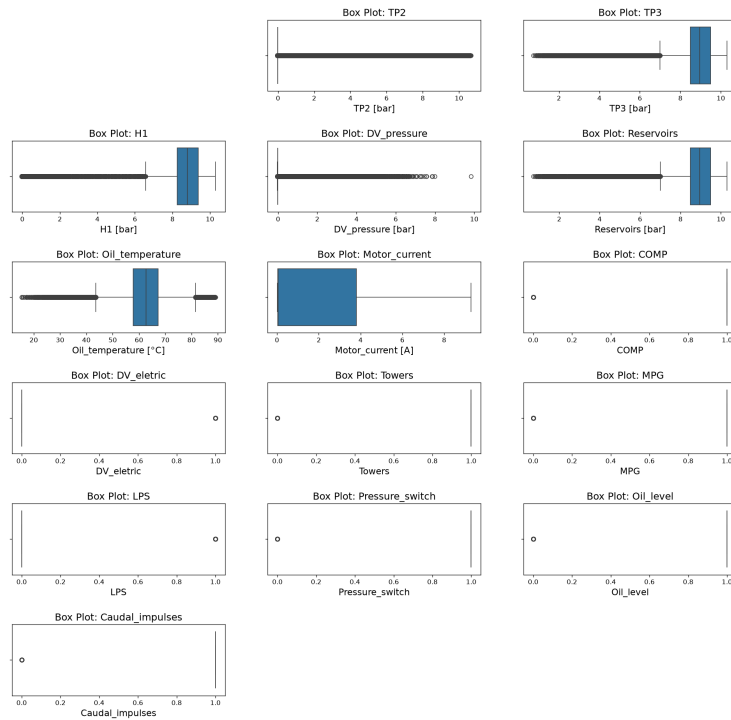
The same seven analog distributions on a log count scale — the active lobe and rare high-pressure events the linear view hides.

Figure 2.4



Pearson and Spearman correlation across the analog signals — TP3 and Reservoirs move together; TP2 and H1 are strongly anti-correlated.

Figure 2.5



Per-signal box plots across all signals — the analog tails and the binary digital channels that inform the labeling scheme.

Labeling — the payoff of investigation

A naive healthy/failure split would poison the healthy baseline. Instead, a six-category scheme (healthy / prefail / infail / postrepair / **degraded** / **invalid**) was built, two of whose categories are *findings*, not givens: a **12-day undocumented degraded span** (a leak running between a failure and its repair) and a **~20-hour frozen-instrument fault** (sensors stuck, machine fine), each carved out by investigation so that windows labeled healthy are genuinely healthy. Base-rate controls throughout reject spurious ‘near-failure’ patterns.

3 Feature Engineering

Sensor readings were aggregated into a gap-aware grid of 4,060 one-hour windows (no window bridges a recording gap). Six feature families were built, each earning its place by whether it separates failures; families that did not (calendar, spectral) were analysed and explicitly excluded rather than quietly kept.

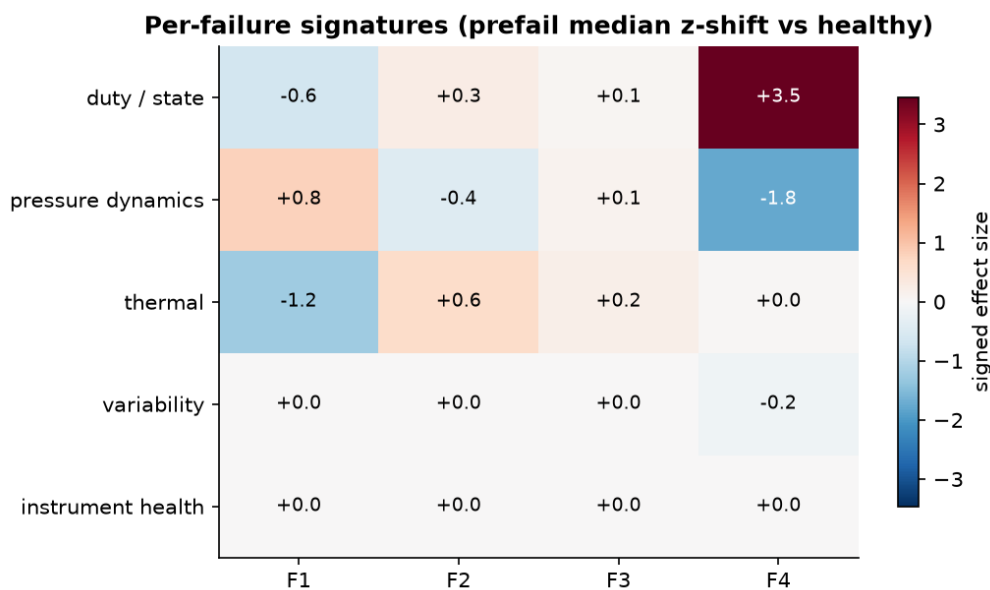
- **Duty & state-occupancy** — the load fraction and the time spent in each motor mode. This is the machine’s baseline workload — the context every other feature is read against, and the axis on which the unique F4 signature stands out most sharply.
- **Pressure dynamics** — a per-idle-stretch pressure-decay slope — the *direct* air-leak meter. When the compressor rests, reservoir pressure should hold; a downward slope during idle is air escaping, measured in physical units rather than inferred.
- **Thermal** — oil level, its trend, and an **oil residual** taken against a healthy oil-temperature-vs-workload baseline. The residual is what separates ‘hot because busy’ from ‘hot because failing’ — the leak’s indirect thermal symptom, isolated from normal load-driven

heating.

- **Variability (cycle dynamics)** — the coefficient of variation and trend of cycle duration. This is the short-cycling signature that rescued the hardest failure (F3), which was invisible to every level- and rate-based feature.
- **Instrument-health** — antiphase and frozen-sensor flags — a **trust axis orthogonal to machine health**. A sensor reading can be wrong without the machine being unwell; separating the two prevents a stuck instrument from masquerading as a fault, or vice versa.

Informative missingness — idle and locked machine states — was modeled explicitly (indicator + fill) rather than dropped, and every feature is scaled against healthy operation so that anomalies read as large deviations.

Figure 3.1



Per-failure effect-size heatmap (prefail median z-shift vs healthy): each feature family vs each failure — the distinct, multidirectional signatures (F4 on duty/state, F1 on thermal, opposite pressure signs).

A recurring lesson

Features were assigned to failures by *measurement*, not expectation. The pressure-decay slope, first computed window-wide, averaged a sawtooth to zero and separated nothing; refitting it per idle-stretch recovered the leak signal. A 'lock-up' feature built to catch one failure caught a different one. The measurement decides — not the intuition that motivated it.

4 Modeling & Results

Three paradigms were compared under one honest protocol — anomaly detection, supervised leave-one-failure-out, and forecast-residual. The method carried forward is the simplest: **zmax**, the maximum standardized deviation of any feature from healthy operation. After the leakage fix described below it is *competitive with, not dominant over* IsolationForest, Mahalanobis, seven supervised classifiers, and forecast-residual — and it is chosen for what those alternatives lack: it needs no training, is fully interpretable (the driving feature names the suspected fault), and gives the longest lead times.

Per-failure detection (the honest scorecard)

Failure	ROC-AUC	Lead time	Verdict
F4 (gradual leak)	0.79	14 h	strongest — flagship case
F1 (idle-then-step)	0.77	32.5 h	detectable, long warning
F3 (erratic cycling)	0.65	9.6 h	marginal — rescued
F2 (short leak)	0.65	—	weak, no early warning
Slow degradation	0.35	—	below chance — not caught

F4 reports precision 0.48 / recall 0.40 at the conservative operating threshold.

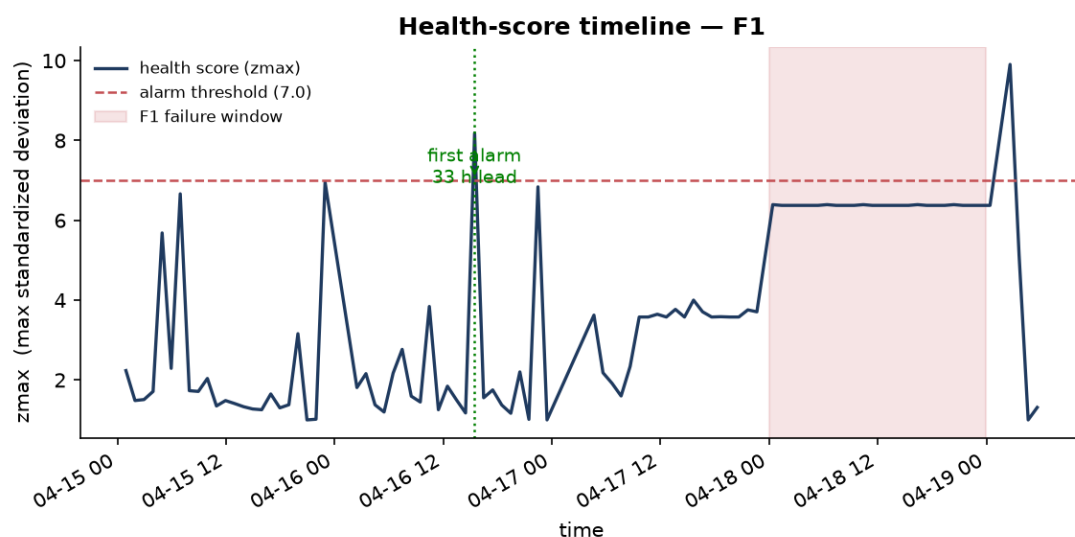
“These are credible predictive-maintenance numbers — real early warning with a real precision / recall / false-alarm tradeoff — not the leakage-driven ~100% accuracy a window-labeled, randomly-split setup produces.”

The integrity centerpiece — a leak found in my own pipeline

Mid-project I found and fixed a **scaler-leakage bug in my own pipeline**: features were z-scored against *all* healthy windows before the train/validation split, leaking validation statistics into the very scores meant to judge them — and a bookkeeping column, `n_samples`, was dominating the score unscaled.

Fixing it **dropped the headline numbers by ~0.1 AUC** (F1 0.86 → 0.77, degraded span 0.58 → 0.35). **Those lower numbers are the real ones**, and they are what this report presents. The common leaky recipe — balancing before splitting, random splits, headline accuracy — produces ~100% and means nothing; this is the exact failure this project was built to avoid, and catching it in my own work is the rigor the project claims.

Figure 4.1

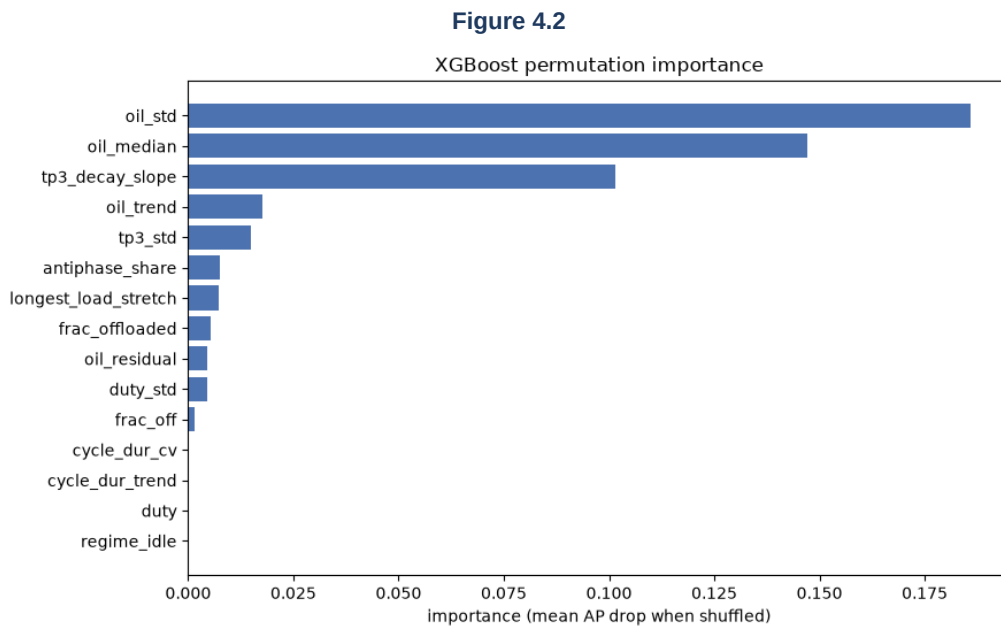


Health-score timeline for F1: the zmax score climbing across the alarm threshold (7.0) — first alarm ~33 h before the failure window (shaded red).

The two findings that matter most

F3 — the hardest case, rescued. F3 was invisible to every level, rate, and even spectral (Lomb-Scargle) feature; it looked genuinely undetectable. It was recovered by the **cycle-dynamics features** (its short-cycling signature) and confirmed by the leave-one-failure-out result that F3's pattern — unlike F4's — is learnable from the other failures.

A failure taxonomy. Leave-one-failure-out training revealed that the two paradigms catch *opposite* failures. Anomaly detection catches *unique* failures (F4: $z_{\max}$ 0.79 ■ supervised 0.48 — it deviates most from healthy but is the hardest to learn from others); supervised transfer catches *shared-pattern* failures (F3: supervised 0.80 > $z_{\max}$ 0.65). They are complementary — a mature system runs both.



Feature importance (prefail-only), XGBoost permutation importance: oil dispersion (oil_std), oil level (oil_median), and the pressure-decay slope (tp3_decay_slope) as the verified predictive signals.

What drives the alarm — verified predictive, not descriptive

Importance was computed on the lead-up only, excluding during-failure windows (which would let the model describe the failure rather than predict it). Two physically-grounded signals dominate: the **pressure-decay slope** (the air leak measured directly) and **oil temperature** (the same leak measured indirectly — the compressor overworks to compensate and heats up). Two sensors, one root cause — cross-confirmed by the independent $z_{\max}$ and XGBoost importance methods.

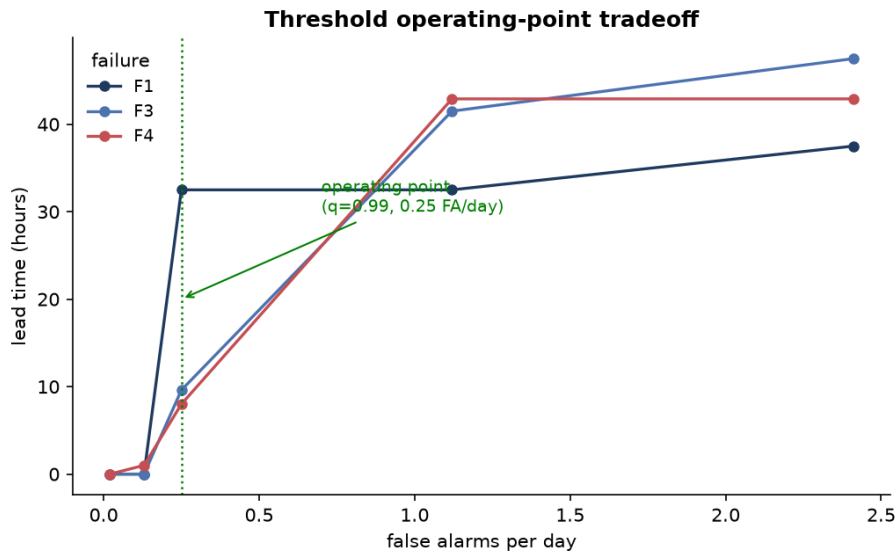
5 From Analysis to Alarm

The pipeline's scoring stage emits, per window, a health score, an alert flag, and the **driving feature** — so an alert is a work order, not a black box:

■ 2020-07-08 18:00 — *health score 23.9 (threshold 7.0), driver: tp3_decay_slope*
→ *suspected air leak, ~8 h to likely failure.*

The alarm threshold is not a fixed choice but a tunable operating point: loosening it buys more lead time at the cost of more false alarms, a tradeoff presented as a menu for the engineer's failure-cost context.

Figure 5.1



Threshold operating-point curve: lead time vs false alarms per day across threshold quantiles, per failure — the operating point ($q=0.99$, 0.25 FA/day) marked.

The live dashboard

A Streamlit real-time health monitor makes the scores operational: a dark industrial theme with a circular health gauge, a health-score timeline with clickable alarm markers and alert/watch threshold lines, live sensor readings in physical units, top-|z| driver bars showing what is pushing the score right now, and a health-vs-trust status gate that separates machine health from instrument health.

Figure 5.2



The FAHM Streamlit dashboard: health gauge, health-and-alarm history against the alert/watch thresholds, live sensor readings, and the top-|z| driver bars.

One command, end to end

```
poetry run python run_pipeline.py --config configs/config.yaml --stage all
```

Raw CSV in, per-failure evaluation and scored alerts out. The runner reproduces the notebook results exactly, verifying that the `src/fahm` package — not the notebooks — is the single source of truth.

6 Honest Limitations

- **F2 detects but does not warn early** — a mild thermal precursor (oil residual +0.64) gives moderate separability but no reliable lead time; it fires only at unusable alarm rates.
- **Slow degradation is below chance** (0.35) — not caught by anomaly detection; better read as a trend than a binary alarm.
- **Low recall by design** at the conservative threshold — the system favors trustworthy early warning over exhaustive flagging; one early true positive suffices to warn.
- **F4's precursor spans weeks** but only crosses the alarm threshold ~14 h out.

The leakage fix belongs here as much as in the results: the honest envelope above exists *because* the ~0.1-AUC drop was accepted rather than hidden. A documented capability envelope is more useful to a maintenance engineer — and more credible to a reviewer — than a suspiciously perfect score.